HPC Project – Parallel Processing of a Scientific Image Dataset

Objective

You are given a **sequential Python application** that processes a collection of scientific images. The provided program follows this pipeline:

`Load image` → `Normalize` → `3x3 mean filter` → `Image statistics` → `Global aggregation`

As the number and size of the images increase, the execution time becomes significant. Your objective is to analyze the provided sequential application and investigate different optimization and parallelization strategies using the techniques studied during the HPC labs.

The provided file `sequential_pipeline.py` must be used as the **reference implementation**. All optimized versions must remain numerically consistent with the sequential baseline.

Team organization

The project is completed in **teams of 4 students**. Each student has a specific technical responsibility.

Student	Role	Main responsibility
1	Performance & Vectorization Engineer	Baseline, profiling, identification of bottlenecks and NumPy/vectorization.
2	Multithreading Engineer	Investigate multithreading, especially I/O versus CPU-bound behavior.
3	Multiprocessing Engineer	Parallelize CPU-intensive work and study speedup and efficiency.
4	MPI Engineer	Distribute the workload using `mpi4py` and evaluate scaling.

All team members must nevertheless understand the complete application and the main conclusions of the project.

Expected work

1. Sequential baseline and profiling

Measure the execution time of the original application and identify where the program spends most of its time. You may use tools such as `time.perf_counter()` and `cProfile`. The team must identify the main performance bottlenecks **before** attempting to optimize or parallelize the program.

2. Vectorization

Investigate whether some explicit Python loops can be replaced by NumPy operations. Compare the execution time before and after vectorization and identify which functions benefit the most.

3. Multithreading

Investigate whether several tasks can be executed using Python threads. In particular, compare the behavior of threading for:

- file loading / I/O operations;
- CPU-intensive processing.

Explain the observed results and the influence of the Python GIL.

4. Multiprocessing

Use several Python processes to process different images concurrently. Test several numbers of workers, for example 1, 2, 4 and 8 processes.

Measure:

$$Speedup(p) = \frac{T_1}{T_p}$$

and:

$$Efficiency(p) = \frac{Speedup(p)}{p}$$

Explain why increasing the number of processes does not necessarily produce proportional acceleration.

5. MPI

Implement a distributed version using `mpi4py`. The dataset must be distributed between MPI ranks, processed independently, and the results must then be gathered or reduced.

Your implementation should use several MPI concepts studied during the lab, for example:

```
Get_rank()
Get_size()
bcast()
scatter()
gather()
reduce()
```

Test your MPI implementation with different numbers of ranks and discuss the observed scaling (weak/strong scaling).

Experimental analysis

Your conclusions must be based on **your own measurements**. Compare the different implementations using a table similar to the one below.

Version	Execution time	Speedup
Sequential		1.00
Vectorized		
Threading		
Multiprocessing		
MPI		

The team must finally answer the following question:

> **Which optimization or parallelization strategy is the most appropriate for this application, and why?**

A combination of several approaches may be proposed.

Deliverables

Source code. At minimum, your repository should contain:

```
sequential_pipeline.py
vectorized_version.py
threading_version.py
multiprocessing_version.py
mpi_version.py
```

Short report. Maximum **5 pages**, including:

1. sequential baseline and profiling;
2. strategies tested;
3. experimental results;
4. performance analysis;
5. final conclusions.

Individual contribution. Each student must clearly identify:

- their role;
- the implementation they developed;
- the experiments they performed;
- their main results;
- what they learned from the project.

Oral defense

Each team will complete a **15-minute oral defense**. During the defense, each student may be asked to:

- explain their implementation;
- justify technical choices;
- explain experimental measurements;
- answer a question about another part of the project;
- make a small live modification to the code.

Using AI tools is allowed as support. However, **each student is responsible for every submitted line of code and must be able to explain, defend and modify it during the oral defense**.

Deadline and contact

The complete project must be submitted **before 23 October 2026**.

If you have questions about the project, or if you want to schedule your **15-minute oral defense**, you may contact me by email: ahmed.el-kerim@safrangroup.com